

BASES DE DATOS NO RELACIONALES

Proyecto Final — Primera Entrega

DrakoBets RAG

Sistema de Consulta Inteligente para Casino Online

Integrantes	Jhoan Sebastian Velez Montes Jaider León Díaz
Materia	Bases de Datos No Relacionales
Entrega	1 — Diseño y Configuración
Fecha	Abril 2026
Repositorio	github.com/londonodaniel230/drakobets_NR

SECCIÓN 1: DOCUMENTO DE ANÁLISIS DEL SISTEMA

1.1 Descripción del Dominio y Contexto del Problema

DrakoBets es una plataforma de apuestas en línea que ofrece dos modalidades principales: juegos de casino (slots, mesas, póker, ruleta y juegos en vivo) y apuestas deportivas sobre competencias y eventos de múltiples disciplinas. La base de conocimiento del sistema está compuesta por descripciones textuales de juegos, reglas de cada modalidad, información de competencias deportivas, perfiles de equipos y jugadores, condiciones de bonos, tipos de apuesta y cuotas.

El sistema RAG se construye sobre la base de datos relacional DrakoBets — implementada en PostgreSQL/Supabase — que contiene 20 tablas distribuidas en tres dominios: casino (juego, tipo_juego, silla, apuesta_casino), apuestas deportivas (apuesta, detalle_apuesta, competencia, evento, deporte, equipo, jugador) y gestión de usuarios (usuario, bono, transaccion). Este modelo relacional preexistente aporta la fuente de datos estructurados que será transformada al modelo documental de MongoDB para habilitar capacidades de búsqueda semántica y generación de respuestas con LLM.

El contexto operacional es el de una plataforma con múltiples usuarios concurrentes que necesitan resolver dudas sobre reglas de juego, estrategias de apuesta, condiciones de bonos y estadísticas de competencias sin navegar manualmente por la documentación. La volumetría estimada para el proyecto es de 100 documentos de texto (juegos y eventos deportivos) y 50 imágenes asociadas (escudos de equipos y fotos de jugadores), con una proyección de crecimiento a 500 documentos en una versión productiva.

1.2 Perfil de Usuarios y Casos de Uso Principales

Perfil	Necesidad de Información	Frecuencia	Consulta Típica
Jugador casual	Entender reglas y probabilidades de un juego de casino antes de apostar	Alta — varias veces al día	¿Cómo se juega el blackjack y cuál es el RTP?
Apostador deportivo	Consultar información de equipos, jugadores y estadísticas de competencias	Media — antes de eventos	¿Qué jugadores tiene el equipo X y cuál es su historial?
Nuevo usuario	Comprender condiciones de bonos y tipos de apuesta disponibles	Baja — durante onboarding	¿Cómo funciona el bono de bienvenida y qué condiciones tiene?
Administrador	Monitorear consultas frecuentes y evaluar calidad del sistema RAG	Baja — reportes periódicos	¿Cuáles son las preguntas más frecuentes de los usuarios?

1.3 Especificación de Requerimientos

ID	Descripción	Componente RAG
----	-------------	----------------

RF-01	El sistema debe ingestar documentos de texto provenientes de las tablas juego, tipo_juego, deporte, competencia, equipo y jugador de DrakoBets en formato JSON.	Ingestion
RF-02	El sistema debe implementar tres estrategias de chunking (fixed-size, sentence-aware y semantic) sobre el campo descripcion_texto de cada documento.	Chunking
RF-03	Cada chunk generado debe almacenarse en MongoDB con el campo estrategia_chunking que identifique la estrategia de origen.	Chunking / Embedding
RF-04	El sistema debe generar embeddings de 384 dimensiones para cada chunk de texto usando el modelo all-MiniLM-L6-v2 (Xenova).	Embedding
RF-05	El sistema debe generar embeddings de 512 dimensiones para imágenes (escudos y fotos) usando el modelo CLIP (clip-vit-base-patch32).	Embedding
RF-06	El endpoint POST /search debe ejecutar búsqueda híbrida combinando \$vectorSearch con filtros de metadatos (categoria, idioma, fecha).	Retrieval
RF-07	El endpoint POST /rag debe recuperar los k chunks más relevantes, construir un prompt con el contexto y generar una respuesta usando Groq + Llama 3.3 70B.	Generacion
RF-08	El sistema debe soportar búsqueda multimodal texto-imagen usando CLIP para encontrar imágenes similares a una descripción de texto.	Retrieval
RF-09	Cada consulta RAG debe guardarse en la colección consultas con la respuesta del LLM, los chunks recuperados y la estrategia usada.	Generacion
RF-10	El sistema debe ejecutar el experimento comparativo de chunking: correr las 10 consultas de prueba sobre las tres estrategias y almacenar los resultados.	Chunking / Retrieval

ID	Requerimiento No Funcional
RNF-01	Rendimiento: el endpoint /rag debe responder en menos de 5 segundos para el 90% de las consultas bajo carga normal.
RNF-02	Escalabilidad: el esquema de MongoDB debe permitir agregar nuevas estrategias de chunking sin modificar la estructura de las colecciones existentes.
RNF-03	Seguridad: las credenciales de MongoDB Atlas y Groq API deben gestionarse exclusivamente mediante variables de entorno (.env), sin hardcoding.
RNF-04	Mantenibilidad: los scripts de inicialización deben ser idempotentes (ejecutables múltiples veces sin errores ni duplicados).
RNF-05	Disponibilidad: se utilizará MongoDB Atlas M0 (capa gratuita) con alta disponibilidad gestionada por Atlas, sin necesidad de infraestructura propia.

1.4 Análisis de Estrategias de Modelado: Embedding vs. Referencing

Entidad	Estrategia	Justificación	Decisión de diseño
tipo_juego	Embedded en juegos	Siempre se lee junto al juego. Catálogo pequeño (< 20 tipos). No se comparte entre usuarios.	Subdocumento tipo_juego: { nombre, descripcion, porcentaje_ganancia }
deporte + competencia	Embedded en eventos_deportivos	Una competencia pertenece a un deporte y se lee siempre con el evento. Jerarquía 1:N natural.	Subdocumentos anidados: evento > competencia > deporte
Imágenes (escudos, fotos)	Referenced (colección imagenes)	Archivos grandes (binarios/URLs). Un escudo puede asociarse a múltiples documentos de equipos.	Campo imagen_id: ObjectId apunta a colección imagenes con embedding CLIP.
bonos + estado_bono	Embedded en usuarios	Un usuario tiene pocos bonos (< 10). Se leen siempre en contexto del usuario.	Array bonos: [{ monto, estado, fecha_vencimiento }]
Chunks de texto	Colección independiente (Referenced)	Un documento genera N chunks por M estrategias. Necesita índice vectorial propio para \$vectorSearch.	Colección chunks con campo doc_id (ref) y estrategia_chunking.
Tablas pivote N:M	Eliminadas (Array en documento padre)	apuestas_competencias y participantexevento son artefactos del modelo relacional. En NoSQL se reemplazan por arrays.	Campo competencias: [ObjectId] dentro de apuestas. Campo participantes: [{tipo, id, nombre}] en eventos.

SECCIÓN 2: DISEÑO DEL ESQUEMA NOSQL

2.1 Definición de Colecciones

Colección	Propósito	Campos Principales	Tipo Relación	Estrategia
juegos	Almacena información de juegos de casino con sus reglas y metadatos. Fuente principal de chunking.	_id, nombre, categoria, descripcion_texto, rtp, volatilidad, idioma, fecha_publicacion, tipo_juego {embedded}, imagen_id (ref)	Referenciada por: chunks, apuestas, silla	Híbrida (metadata embedded, imagen referenced)
eventos_dep ortivos	Almacena eventos con su competencia y deporte embebidos. Segunda fuente de chunking.	_id, fecha, ciudad, competencia {embedded}, deporte {embedded}, participantes [], descripcion_texto, imagen_id (ref)	Referenciada por: chunks, apuestas	Híbrida (competencia/deporte embedded, imagen referenced)
chunks	Fragmentos de texto generados por las tres estrategias de chunking. Contiene los vectores para búsqueda semántica.	_id, doc_id (ref), chunk_index, estrategia_chunking, chunk_texto, embedding[], num_tokens, modelo, fecha_ingesta	Referencia a: juegos, eventos_depo rtivos	Referenced (doc_id apunta al documento padre)
imagenes	Imágenes de escudos de equipos y fotos de jugadores con embeddings CLIP para búsqueda multimodal.	_id, url, descripcion_alt, embedding_clip[512], formato, ancho, alto, doc_ref (opcional)	Referenciada por: juegos, eventos_depo rtivos	Referenced (independiente, compartida)
usuarios	Datos de usuarios con bonos y transacciones embebidos.	_id (cedula), nombres, apellidos, correo, bonos [] {embedded}, transacciones [] {embedded}	Referenciada por: apuestas, consultas	Embedded (bonos y transacciones dentro del usuario)
apuestas	Apuestas de casino y deportivas unificadas en una colección con tipo discriminador.	_id, tipo (casino/dep.), usuario_id (ref), monto, fecha, resultado, detalles [] {embedded}, competencias [] (ref)	Referencia a: usuarios, juegos, eventos_depo rtivos	Embedded (detalles) + Referenced (relaciones)

consultas	Historial de consultas RAG con respuestas del LLM y chunks usados. Sirve para el experimento de chunking.	_id, query_texto, respuesta_llm, estrategia_usada, chunks_recuperados [], tiempo_respuesta_ms, fecha	Sin relación directa (log independiente)	Embedded (historial autocontenido)
-----------	---	--	--	------------------------------------

2.2 Ejemplos de Documentos

Colección: juegos

```
{  "_id": ObjectId("..."),  "nombre": "Blackjack Clásico",  "categoria": "mesa",  "descripcion_texto": "El Blackjack es un juego de cartas en el que el objetivo es obtener una mano cuyo valor sea lo más cercano posible a 21 sin pasarse. El jugador compite contra el crupier. Se reparten dos cartas a cada participante...",  "rtp": 99.5,  "volatilidad": "baja",  "idioma": "es",  "fecha_publicacion": ISODate("2024-01-15"),  "tipo_juego": {    "nombre": "mesa",    "descripcion": "Juegos de mesa con crupier en vivo",    "porcentaje_ganancia": 2.5  },  "metadatos": {    "apuesta_min": 1,    "apuesta_max": 5000,    "jackpot": false,    "etiquetas": ["popular", "estrategia"]  },  "imagen_id": ObjectId("...")}
```

Colección: chunks

```
{  "_id": ObjectId("..."),  "doc_id": ObjectId("ref al juego padre"),  "chunk_index": 2,  "estrategia_chunking": "sentence-aware",  "chunk_texto": "El jugador compite contra el crupier. Se reparten dos cartas a cada participante al inicio de la mano.",  "embedding": [0.023, -0.117, 0.045, ...],  // 384 dimensiones  "num_tokens": 28,  "modelo": "all-MiniLM-L6-v2",  "fecha_ingesta": ISODate("2026-04-25")}
```

2.3 Estrategias de Indexación

Colección	Campo(s) Indexado(s)	Tipo de Índice	Justificación
juegos	{ fecha_publicacion: 1, idioma: 1 }	Compuesto	Requerido por el enunciado. Optimiza filtros como 'juegos en español de 2024'.
juegos	{ descripcion_texto: 'text', nombre: 'text' }	Texto (Atlas Search)	Habilita búsqueda full-text con \$search. Permite \$searchBeta con fuzzy matching.
chunks	{ embedding: 'knnVector' } — 384 dims	Vector Search (Atlas)	Índice principal del sistema RAG. Habilita \$vectorSearch con similitud coseno para recuperar los k chunks más relevantes.
imagenes	{ embedding_clip: 'knnVector' } — 512 dims	Vector Search (Atlas)	Búsqueda multimodal texto-imagen con CLIP. 512 dimensiones corresponden a clip-vit-base-patch32.

chunks	{ estrategia_chunking: 1, doc_id: 1 }	Compuesto	Permite filtrar chunks por estrategia durante el experimento. Fundamental para comparar fixed-size vs sentence-aware vs semantic.
--------	--	-----------	---

2.4 Reglas de Validación de Esquema

Colección: juegos

```
db.createCollection("juegos", { validator: { "$jsonSchema": {  
  "bsonType": "object", "required":  
    ["nombre", "categoria", "descripcion_texto", "idioma", "fecha_publicacion"],  
  "properties": { "nombre": { "bsonType": "string", "minLength": 3 },  
    "categoria": { "bsonType": "string", "enum":  
      ["slot", "mesa", "poker", "ruleta", "en_vivo"] }, "rtp": {  
      "bsonType": "number", "minimum": 0, "maximum": 100 }, "volatilidad": {  
      "bsonType": "string", "enum": ["baja", "media", "alta"] }, "idioma": {  
      "bsonType": "string", "enum": ["es", "en", "pt"] } } } },  
  validationAction: "error" })
```

Justificación de campos requeridos: nombre y descripcion_texto son los campos que alimentan el chunking — sin ellos el documento no puede procesarse. categoria permite aplicar filtros híbridos en \$vectorSearch. idioma y fecha_publicacion son necesarios para el índice compuesto obligatorio del enunciado.

Colección: chunks

```
db.createCollection("chunks", { validator: { "$jsonSchema": {  
  "bsonType": "object", "required":  
    ["doc_id", "chunk_index", "estrategia_chunking",  
    "chunk_texto", "embedding", "modelo"], "properties": {  
    "estrategia_chunking": { "bsonType": "string", "enum":  
      ["fixed-size", "sentence-aware", "semantic"] }, "embedding": {  
      "bsonType": "array", "minItems": 384, "maxItems": 512  
    }, "chunk_index": { "bsonType": "int", "minimum": 0 } } } },  
  validationAction: "error" })
```

Justificación: estrategia_chunking es el campo central del experimento — el enum garantiza que solo se ingresen las tres estrategias definidas y previene errores de escritura. El campo embedding con minItems:384 asegura que no se almacenen vectores incompletos que contaminarían los resultados de \$vectorSearch.

SECCIÓN 3: CONFIGURACIÓN DEL ENTORNO

3.1 Especificación del Entorno

Se utiliza MongoDB Atlas (Opción A del enunciado) con cluster gratuito M0 en AWS us-east-1. Esta elección se justifica por el acceso nativo a Atlas Vector Search — necesario para el índice knnVector — sin configuración manual adicional, y por el monitoreo integrado que facilita el análisis del experimento de chunking.

Herramienta / Librería	Versión	Propósito en el proyecto
MongoDB Atlas	M0 (MongoDB 7.0)	Base de datos principal. Cluster gratuito con Vector Search nativo.
Node.js	18+	Runtime del backend (Express + Groq SDK + Xenova).
Python	3.10+	Scripts de generación de embeddings de texto e imágenes.
MongoDB Driver (Node)	mongodb ^6.x	Conexión nativa al cluster Atlas desde el backend.
@xenova/transformers	^2.x	Embeddings locales con all-MiniLM-L6-v2 (384 dims) en Node.js.
sentence-transformers (Python)	^2.x	Alternativa Python para embeddings de texto en scripts de ingesta.
transformers / OpenCLIP (Python)	^4.x	Embeddings de imágenes con clip-vit-base-patch32 (512 dims).
LangChain (Python)	^0.2.x	text_splitter para estrategias fixed-size y sentence-aware.
semantic-chunkers (Python)	^0.0.x	Chunking semántico basado en similitud coseno entre oraciones.
Groq SDK (Node.js)	groq-sdk ^0.x	Integración con Llama 3.3 70B Versatile como LLM generativo.
Express.js	^4.x	Framework HTTP para los endpoints POST /search y POST /rag.
dotenv	^16.x	Gestión de variables de entorno (MONGO_URI, GROQ_API_KEY).
MongoDB Compass	Última estable	Exploración visual del cluster y validación de documentos.

3.2 Información del Clúster

Parámetro	Valor
Nombre del clúster	Cluster0 — DrakoBets
Región / Proveedor de nube	AWS us-east-1 (N. Virginia)
Nombre de la base de datos	Proyecto
Versión de MongoDB confirmada	MongoDB 7.0 (Atlas M0)

Método de autenticación	SCRAM-SHA-256 con usuario dedicado al proyecto
Colecciones creadas	juegos, eventos_deportivos, chunks, imagenes, usuarios, apuestas, consultas

3.3 Script de Conexión y Verificación

```
// backend/src/config/db.js
const { MongoClient, ServerApiVersion } =
require('mongodb');require('dotenv').config();let client;let db;async function
connectDB() { if (db) return db; // Reutiliza conexión existente client = new
MongoClient(process.env.MONGO_URI, { serverApi: { version:
ServerApiVersion.v1, strict: true, deprecationErrors: true } }); await
client.connect(); // Verificar conexión exitosa con ping await
client.db("admin").command({ ping: 1 }); console.log("Conectado a MongoDB
Atlas - DrakoBets"); db = client.db(process.env.DB_NAME); return
db;}module.exports = { connectDB };
```

3.4 Script de Inicialización (init_db.js)

El script es idempotente: usa createCollection con validación solo si la colección no existe, y createIndex con la opción { background: true } que no falla si el índice ya existe.

```
// scripts/init_db.js
require('dotenv').config({ path: '../backend/.env'
});const { MongoClient } = require('mongodb');async function init() { const
client = new MongoClient(process.env.MONGO_URI); await client.connect();
const db = client.db(process.env.DB_NAME); const existing = (await
db.listCollections().toArray()).map(c => c.name); // — Colección juegos con
schema validation if (!existing.includes('juegos')) { await db.createCollection('juegos', {
validator: { "$jsonSchema": { "bsonType": "object",
"required":
["nombre","categoria","descripcion_texto","idioma","fecha_publicacion"],
"properties": { "categoria": { "enum":
["slot","mesa","poker","ruleta","en_vivo"] }, "idioma": { "enum":
["es","en","pt"] } } } }); console.log('Coleccion
juegos creada'); } // — Colección chunks con schema validation
if (!existing.includes('chunks')) { await
db.createCollection('chunks', { validator: { "$jsonSchema": {
"bsonType": "object", "required":
["doc_id","chunk_index","estrategia_chunking",
"chunk_texto","embedding","modelo"], "properties": {
"estrategia_chunking": { "enum":
["fixed-size","sentence-aware","semantic"] } } }
}); console.log('Coleccion chunks creada'); } // — Colecciones sin
validación estricta for (const col of
['eventos_deportivos','imagenes','usuarios','apuestas','consultas']) { if
(!existing.includes(col)) { await db.createCollection(col);
console.log('Coleccion ' + col + ' creada'); } } // — Índice compuesto
obligatorio await
db.collection('juegos').createIndex( { fecha_publicacion: 1, idioma: 1 },
{ name: 'idx_fecha_idioma', background: true } ); // — Índice de texto
await
db.collection('juegos').createIndex( { descripcion_texto: 'text', nombre:
'text' }, { name: 'idx_texto_juegos', background: true } ); // — Índice
compuesto para experimento de chunking await
db.collection('chunks').createIndex( { estrategia_chunking: 1, doc_id: 1 },
{ name: 'idx_estrategia_docid', background: true } ); console.log('Índices
creados. Recuerda crear los índices vectoriales en Atlas UI.');
```

Nota: los índices knnVector (Vector Search) deben crearse desde la interfaz de Atlas UI o mediante la API de Atlas, ya que el driver de Node.js no soporta su creación directa. Se configuran así: colección chunks con campo embedding, 384 dimensiones, similitud coseno; colección imagenes con campo embedding_clip, 512 dimensiones, similitud coseno.

SECCIÓN 4: PLAN DE EXPERIMENTO DE CHUNKING

4.1 Marco Conceptual del Experimento

a) ¿Por qué el chunking afecta la calidad del sistema RAG?

En un sistema RAG, el modelo de lenguaje no recibe el documento completo sino los k fragmentos más similares a la consulta. Si un chunk corta a la mitad la explicación de una regla de juego, el contexto que llega al LLM estará incompleto y la respuesta será vaga o incorrecta. Por el contrario, si el chunk es demasiado largo incluye información de múltiples temas y el vector de embedding promedia semánticamente todo el contenido, reduciendo la precisión de la búsqueda por similitud.

b) Métrica de comparación

Se usará evaluación cualitativa percibida (escala 1–5) por cada consulta de prueba, complementada con métricas cuantitativas: número de chunks generados por documento, longitud promedio en tokens, y diversidad de chunks recuperados por consulta. Si el tiempo lo permite, se implementará RAGAS para obtener faithfulness, answer_relevancy y context_recall de forma automática (nota extra).

c) Hipótesis

Hipótesis principal: para el dominio de DrakoBets, la estrategia sentence-aware producirá los mejores resultados. Justificación: los textos principales (descripcionjuego, reglasjuego, descripciondeporte) son narrativos y están escritos en oraciones completas con sentido propio. El chunking por oración respeta la unidad semántica mínima del texto de casino, evita cortar definiciones de términos técnicos a la mitad, y produce chunks con semántica coherente sin requerir el costo computacional del chunking semántico. El fixed-size probablemente produzca los peores resultados por cortar dentro de oraciones.

4.2 Estrategias de Chunking Seleccionadas

Estrategia	Tamaño (tokens)	Overlap	Criterio/Separador	Justificación para DrakoBets
Fixed-size	256 tokens	32 tokens	Corte por conteo de tokens	Baseline del experimento. Simple y predecible. Se espera que corte en medio de reglas de juego, lo que produce contexto incompleto.
Sentence-aware	Máx. 5 oraciones	1 oración	Límites de oraciones (spaCy / NLTK)	Estrategia candidata principal. Los textos de casino y deportes son narrativos; respetar oraciones preserva definiciones completas de reglas y características.
Semantic	Variable (por similitud)	0 (por umbral)	Umbral coseno 0.75–0.85	Agrupar oraciones por tema. Útil para documentos largos de deportes con múltiples secciones (historial, plantilla, estadísticas).

4.3 Conjunto de 10 Consultas de Prueba

Las consultas están diseñadas para cubrir los tipos de contenido de DrakoBets y evaluar diferencias entre estrategias en consultas cortas, largas, técnicas y de contexto mixto:

#	Consulta de Prueba	Tipo	Colección Objetivo
1	¿Cuáles son las reglas básicas del blackjack y cómo se gana?	RAG — reglas	juegos (mesa)
2	¿Qué es el RTP y qué slots tienen el RTP más alto disponible?	Semántica — métrica	juegos (slot)
3	Explica cómo funciona la ruleta europea y cuáles son las apuestas disponibles.	RAG — explicación larga	juegos (ruleta)
4	¿Qué diferencia hay entre apuesta simple y apuesta combinada?	RAG — comparativo	tipo_apuesta / eventos
5	¿Qué jugadores conforman el equipo con más victorias en la competencia actual?	Semántica — deportes	eventos_deportivos
6	¿Cómo funciona el bono de bienvenida y qué condiciones debo cumplir para usarlo?	RAG — condiciones	usuarios (bonos)
7	¿Qué estrategias básicas recomienda el sistema para ganar en el póker?	RAG — estrategia	juegos (poker)
8	¿Cuáles son los deportes disponibles para apostar y en qué regiones se juegan?	Semántica — catálogo	eventos_deportivos (deporte)
9	Muéstrame imágenes de equipos de fútbol disponibles para apostar.	Multimodal (CLIP)	imagenes (escudos)
10	¿Qué juegos de casino tienen jackpot progresivo y cómo funciona?	Híbrida — filtro + RAG	juegos (jackpot: true)

4.4 Protocolo del Experimento

- Ejecutar el script de ingesta (ingest.py) que genera los tres conjuntos de chunks desde las mismas fuentes y los almacena en la colección chunks con su respectivo campo estrategia_chunking.
- Para cada una de las 10 consultas, ejecutar POST /rag tres veces — una por estrategia — pasando el parámetro estrategia en el body del request.
- Registrar en la colección consultas: la respuesta generada, los IDs de chunks recuperados, la estrategia usada y el tiempo de respuesta.
- Completar la tabla comparativa evaluando la calidad de cada respuesta en escala 1–5 según criterios de completitud, precisión factual y coherencia.
- Calcular estadísticas descriptivas: número de chunks por documento y longitud promedio por estrategia usando aggregation pipeline (\$group, \$avg, \$count).
- Redactar la conclusión argumentada comparando los resultados con la hipótesis planteada.